

Detection of Poisoning Attacks in Federated Learning with Non-IID data

Divya Bhatt*

Department of Computer Science
The University of Oklahoma
Norman, USA
Divya.Bhatt-1@ou.edu

Le Gruenwald

Department of Computer Science
The University of Oklahoma
Norman, USA
ggruenwald@ou.edu

Abstract—Federated Learning (FL) is a decentralized approach to training machine learning models while preserving the privacy of training data. In FL, the training data remains on local clients, and only model updates are shared with a central server to collaboratively learn a global model. However, this decentralized nature makes FL vulnerable to poisoning attacks, where malicious clients can compromise the integrity of the model by submitting manipulated updates. To defend against such attacks, we propose a new defense algorithm called RBF, based on Radial Basis Functions. The RBF defense algorithm constructs a similarity matrix between client model updates and then analyzes this matrix to detect anomalous patterns that may indicate malicious behavior. Identified malicious clients are excluded from the aggregation process to enhance the robustness and prediction accuracy of the global model. We evaluate RBF on three real-world datasets using four different learning models. Its performance is compared against two existing kernel-based defense algorithms: KPCA with KMeans (Kernel Principal Component Analysis with clustering) and LoMar (Local Malicious Factor using Kernel Density Estimation). Experimental results demonstrate that RBF outperforms both baselines across standard evaluation metrics, including Precision, Recall, F1-Score, Attack Success Rate, and Global Prediction Accuracy. Additionally, RBF achieves the lowest execution time with two models and comparable performance with the other two models. Further experiments varying the number of clients, the proportion of malicious participants, and data heterogeneity (IID vs Non-IID) show that RBF maintains stable and competitive performance under diverse conditions. These findings establish RBF as an effective and efficient defense against poisoning attacks in federated learning systems.

Keywords—federated learning, data poisoning, non-IID

I. INTRODUCTION

Federated Learning, a decentralized form of machine learning, was introduced in 2016 to harness the vast amounts of training data on client devices such as smart phones and Internet of Things (IoT) devices [1]. While this decentralized data holds immense potential for improving model performance, it is often privacy sensitive and impractical to transfer to a centralized location due to communication bandwidth limitations and regulatory constraints. FL addresses these challenges by enabling clients to retain their data locally and collaboratively train a shared global model. Under the orchestration of a central server, each client trains a local model on its private data and sends only the model updates to the server. The server then aggregates these updates to refine the global model, without ever accessing the raw client data.

The number of clients in a federated learning system can range from a small group to hundreds of thousands, particularly in mobile and edge device settings [1]. These clients may vary widely in nature, encompassing entities such

as large organizations, mobile phones, or individual sensors. The training data across clients can exhibit significant heterogeneity—it may be independent and identically distributed (IID) or non-IID to varying degrees, and often presents challenges such as high class imbalance, differing feature distributions, and varying data volumes. This variability introduces complexities in both model convergence and robustness, making the design of effective aggregation and defense mechanisms critical in FL systems.

While FL enhances data privacy by keeping training data on client devices, it inadvertently expands the attack surface, particularly at the client level. Given the potentially vast number of participants in an FL system, it becomes increasingly feasible for adversaries to insert malicious clients. These compromised clients can engage in poisoning attacks by manipulating their local training data or crafting malicious model updates to degrade the performance of the global model. Importantly, such attacks are often designed to be subtle—allowing the global model to converge—thus evading detection mechanisms [5,10,11]. As a result, it is critical to design and deploy robust defense algorithms that can accurately detect and mitigate malicious client behavior before it significantly impacts the integrity and accuracy of the global model.

Several defenses against client-side poisoning attacks in FL have been proposed in the literature [6,7,10,11,16,17,19]. A widely adopted approach involves the use of distance-based measures, such as Euclidean distance, to distinguish between benign and malicious client updates. Among the commonly used defenses are Trimmed Mean, Median, PCA with KMeans, KRUM, and COOMED [36]. An extension of PCA, KPCA with KMeans [36], introduces nonlinearity through kernel methods and then uses clustering to distinguish between benign and malicious updates. Another notable technique is LoMar [30], which leverages Kernel Density Estimation and K-Nearest Neighbors (KNN) to compute a local malicious score for each client. The clients with high malicious scores are excluded from the aggregation at the server.

Despite their utility, many of these defenses—particularly those relying on Euclidean distances—are fundamentally linear and fail to capture the nonlinear relationships inherent in high-dimensional model updates. While KPCA introduces nonlinearity via kernel functions, it performs linear dimensionality reduction by projecting data onto directions of maximum variance, potentially discarding important discriminative information. Moreover, KMeans clustering, used in both PCA and KPCA variants, is highly sensitive to the choice of the hyperparameter K (the number of clusters). It is most effective when clusters are spherical and well-separated, but its performance degrades on irregularly shaped

or overlapping clusters—conditions that are common in the heterogeneous and noisy environment of FL. The LoMar algorithm attempts a more fine-grained analysis by decomposing each model update into label-specific components, applying KNN to each part, and using Kernel Density Estimation to assess maliciousness. While this approach can provide detailed per-label insights, it becomes computationally intensive, particularly when the number of output labels is large or when client models are complex.

Taken together, these limitations suggest that while existing methods offer useful insights, their effectiveness and scalability in real-world FL scenarios remain constrained, motivating the need for more adaptable, nonlinear, and computationally efficient defense strategies.

To address the limitations of existing defenses, we propose a novel defense algorithm called RBF, based on Radial Basis Functions. This method employs nonlinear kernel functions to capture the intricate, nonlinear relationships between client model updates in the form of a similarity matrix. In contrast to KPCA, which reduces the similarity matrix by projecting onto axes of highest variance, the RBF method directly operates on the full similarity matrix. This preserves richer structural information, allowing for more effective detection of anomalous patterns indicative of malicious behavior.

The main contributions of this work are summarized as follows:

- We introduce a new RBF-based defense algorithm for mitigating client-side poisoning attacks in federated learning, and compare its performance against two state-of-the-art kernel-based techniques: KPCA with KMeans and LoMar.
- We demonstrate that RBF achieves superior performance in terms of standard malicious client detection metrics (Precision, Recall, F1-Score, and Attack Success Rate) and enhances the global model's prediction accuracy on both IID and non-IID datasets.
- We show that RBF has favorable computational efficiency, achieving the lowest execution time in two of the four learning models tested, and outperforming LoMar while remaining comparable to KPCA with KMeans for the remaining two models.

The rest of the paper is organized as follows: Section II introduces the FL learning framework, the threat model, and the relationship between poisoning attacks and global model degradation, Section III presents the details of the proposed RBF algorithm, Section IV describes the experimental setup including datasets, models, and evaluation metrics, and Section V provides experimental results. Section VI is a discussion of the significance of the experimental results, and Section VII concludes with a summary and directions for further work.

II. PRELIMINARIES

A. System Model

A typical FL system consists of a global server, and several (possibly heterogenous) clients which communicate with each other. Each client retains its own data which might be privacy-sensitive, non-IID or imbalanced. Fig. 1 shows an example of such a system.

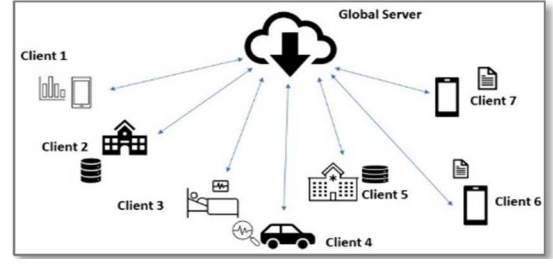


Fig. 1. Typical Federated Learning System

The global server coordinates the learning and clients and server communicate to share the global model and client model updates. The steps that occur in a typical FL system are detailed below. In a traditional FL system, the number of iterations and/or the number of clients could run into several thousand [1,2].

Step 1: The global server initializes a global model $\mathbf{w}^0$ that it is seeking to converge and use for prediction. Total number of iterations is set as T . K is the total number of clients, C is the percentage of clients selected in each iteration.

Step 2: The server selects $N=K*C$ clients for training based on criteria such as availability. The server begins the first iteration at $t=1$ where $t=1..T$ and delivers the global model $\mathbf{w}^{t-1}$ to each of the selected clients.

Step 3: Each selected client i where $i=1..N$ uses the model $\mathbf{w}^{t-1}$ and the local training dataset $\mathbf{D}_i$ to produce an updated local model as $\mathbf{w}_i^t = \mathbf{w}^{t-1} - \eta_i \nabla L_i(\mathbf{w}^{t-1}, \mathbf{D}_i)$ where η_i is the learning rate of client i , ∇L_i is the gradient of the optimization loss function at client i .

Step 4: Clients might conduct a few epochs of local training as described in **Step 3** before sending model updates $\mathbf{u}_i = \mathbf{w}_i^t - \mathbf{w}^{t-1}$ back to the server. The server aggregates all client updates (and might perform other processing such as sanitizing the updates before aggregation) using an aggregation algorithm designed for FL such as FedAvg [1]. This results in an updated model at the server:

$$\mathbf{w}^t = \mathbf{w}^{t-1} + \frac{1}{\sum_{i=1}^N n_i} \sum_{i=1}^N n_i \mathbf{u}_i$$
 where n_i is the number of training samples at client i .

Step 5: One iteration is completed and **Steps 2-4** repeat until the model converges or maximum number of iterations is reached and the learning terminates.

B. Threat and Attack Model

There are numerous ways in which adversarial attacks can be executed in a FL environment [35]. Potential targets include the clients, their local data, local models, the central server, server-side data and models, or even the communication network. In this paper, we focus specifically on **client-side poisoning attacks** that occur during the training phase of the FL process.

The primary objective of a malicious actor in such an attack is to degrade the performance of the global model, reducing its prediction accuracy compared to an un-attacked baseline. Typically, the adversary seeks to induce targeted or untargeted misclassifications in the global model while maintaining sufficient consistency in updates to allow the global model to converge—thus avoiding detection. These stealthy attacks are particularly dangerous because they

compromise the integrity of the global model without raising suspicion.

1) *Malicious client subset*: A subset of total number of clients (not greater than 40%) are malicious. If there is a malicious attack, at least one client is affected. These malicious clients have access to training data and the entire client training process, including client models. Label flipping attacks are conducted on malicious clients, and the number of flipped labels in each malicious client could range from 1 to all output labels. In our experiments, all output labels were flipped in malicious clients.

2) *Training data manipulation*: The malicious actor manipulates the training process by using auxiliary data (for e.g. benign data with labels flipped to be used in the attack). After creating a malicious subtask, techniques to improve effectiveness of the attack such as explicit boosting and alternate minimization [3,36] are used.

3) *Attack optimization strategies*: **Explicit boosting** is performed by the malicious client by first training on the legitimate data mixed with the auxiliary data (valid training data with labels flipped) to determine the desired malicious client update. The malicious actor then computes the difference between this malicious update and the global model (available on the client at the start of the iteration), multiplies it by a boosting factor, and then sends it to the server. This is done to ensure the malicious update still has effect after aggregation with the updates from the benign clients. In **alternate minimization** the distance between the malicious update and the benign update (which can be computed by the malicious client with its portion of the benign training data) is minimized as a stealth strategy. Then the malicious client alternates between using the stealth strategy and sending regular benign client updates during the training process.

C. Attack Assumptions

- In a poisoning attack on clients, the server or the network is not targeted directly, but only the global model on the server is affected.
- The poisoning attacks are on the clients who do not communicate or share information or data with each other.
- None of the attacks are specific to a particular dataset, learning model, loss function or optimization method.
- The malicious actor controls at least one client during an attack.
- Malicious clients have access to legitimate training data, as well as auxiliary data that is needed to create a malicious sub-task, as well as the learning process.
- The result of the client-side attack produces poisoned client model updates that are sent to the server.

D. Relationship between attacks and model updates

Since label-flipping attacks affect updates sent to the server in some fashion, they are considered a special case of direct model update attacks [35]. Some literature suggests training data attacks might be weaker than direct model update attacks [35] but the number of clients compromised in a training data attack ("participation rate") plays a significant role in the strength of the attack, as well as the amount of

training data that is affected and any explicit boosting or stealth strategies. Both attacks result in poisoned client model updates and this research discusses a robust defense of the same.

III. DEFENSE DETAILS

There are two main types of traditional defenses against poisoning attacks:

- Removing poisoned updates by analysis of remote updates at the server
- Developing tolerating aggregating functions at the server that allow the server to achieve the FL objective with the poisoned updates

Even a baseline malicious label-flipping attack can bypass most of the naïve defenses by adding optimization strategies such as boosting and stealth metrics [4,6].

In this research, a defense method called RBF, based on Radial Basis Functions, is introduced which identifies poisoned updates at the server and removes them from the aggregation process before the global model is affected. The defense is experimentally tested with both stealth metrics and explicit boosting to ensure its robustness. RBF is compared in performance with two existing defenses – KPCA with KMeans and LoMar - which are also based on kernel methods and aim to defend against label flipping attacks in the presence of stealth metrics and boosting.

A. RBF Algorithm Details

The RBF technique consists of developing a similarity matrix pairwise between all the remote updates in each server iteration using a kernel function $\mathbf{K}$ as shown below. In our research, a Gaussian kernel was used, however, any other kernel function can be used.

$$\mathbf{K}(\mathbf{u}_i, \mathbf{u}_j) = \exp\left(-\frac{\|\mathbf{u}_i - \mathbf{u}_j\|^2}{2\sigma^2}\right) \quad (1)$$

where $\mathbf{u}_i, \mathbf{u}_j$: client model updates for $i^{\text{th}}, j^{\text{th}}$ clients

σ : bandwidth of the kernel

In this matrix, each row and column correspond to a specific client, with values indicating how similar their updates are to one another. Figure 2 illustrates an example of such a similarity matrix for 8 clients, where Client 4 is malicious. A similarity score of 1 indicates that the updates are highly similar (i.e., nearly identical vectors), while lower scores reflect greater dissimilarity between updates. These anomalies can be detected by analyzing the structure of the similarity matrix and identifying clients whose patterns of similarity deviate significantly from the benign group. In RBF, the distinction between malicious and benign updates is determined using the method described in Section III-B. During aggregation, only the benign client updates, as identified by the RBF-based filtering method, are included in the FedAvg algorithm. Updates from clients classified as malicious are excluded, thereby mitigating their influence on the global model and enhancing the system's robustness to poisoning attacks.

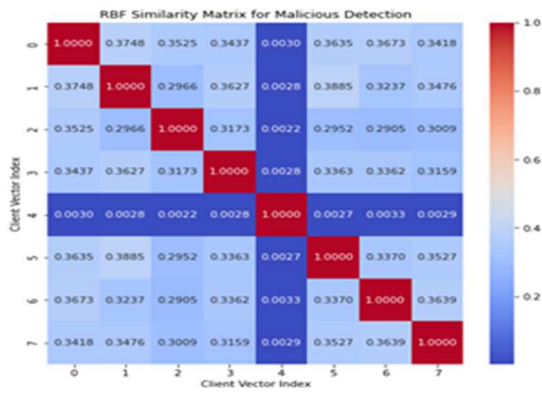


Fig. 2. RBF Similarity Matrix

B. Determining RBF Similarity Score Threshold

Determining the correct threshold for similarity scores to distinguish between benign and malicious updates at the server is a challenging task. In addition to being accurate, the technique needs to be computationally efficient, as the operation is performed with every iteration at the server and typically the number of iterations will be a large number in several thousand with high dimensional vectors. Starting with a similarity matrix, following are the steps used in the RBF algorithm to determine the threshold between the benign and malicious scores.

1) Summation of Similarity Scores

A summation vector is created with the sum of all the similarity scores along the columns for each client, resulting in a single vector for all clients with each score being the sum of the similarity scores of that vector with all other client vectors. Fig 3 shows the summation vector for the similarity matrix of Fig 2.

2) Detecting Malicious Clients

An analysis of the summed similarity matrix shown in Fig. 3 showed some statistical patterns that were used to identify malicious updates. There were three main situations that were present – one single malicious client update present, multiple malicious client updates present, and no malicious client updates present (due to a subset of the possible clients randomly being picked in each iteration). The malicious client updates did present different values from the rest of the benign updates, mostly very low values due to lack of similarity, or else a very high value due to stealth metrics, explicit boosting and alternate minimization. Techniques such as clustering to identify the malicious updates produced fair results, with slightly lower metrics compared to KPCA with KMeans and LoMar methods.

The following two-pronged technique resulted in higher performance metrics:

- The z-score of each value in the similarity summation matrix is calculated. Malicious updates tended to have almost identical or very similar z-scores (due to the poisoning process being similar in malicious clients and all output labels being targeted in the attacks). The z-scores of the benign clients had greater separation in values. If multiple malicious clients were present, then very similar or identical z-scores identified them.

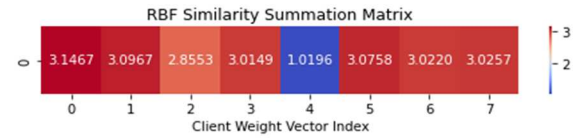


Fig. 3. Summation Matrix for Similarity Scores

- In addition, an enhanced technique was used to identify and double check the presence of one or many malicious clients. First the median value of the summation array is calculated, then the median of the absolute value of deviations of all values from the median (Median Absolute Deviation) is calculated. These are used to calculate the modified z-score for each value in the summation matrix using equation (2).

$$\text{Modified z-score} = \frac{0.6745 * (\text{value} - \text{median})}{\text{Median Absolute Deviation}} \quad (2)$$

If any client shows high modified z-scores, it is likely to be an outlier. To confirm that this is indeed an outlier, another check is performed. The suspected outlier is removed from the summation matrix and the mean modified z-score of the rest of the summation vector is calculated. If the difference of the modified z-score of the outlier and the mean modified z-score of the summation matrix without the outlier is at least 1.5, it is considered a true outlier and removed as malicious.

- In the situation where there are no malicious clients in the similarity summation matrix, all benign modified z-scores will show the same variation which is much less than the variation when malicious clients are present.

IV. EXPERIMENTAL SETUP

In this section, the setup of the experiments are described including dataset details, models used for learning, and metrics used for performance evaluation.

A. Simulation Model

The FL model used for experiments is a modified version of the online repository made available by Bhagoji [3] using 3 different datasets – Fashion-MNIST, CIFAR-10, UCI Adult CENSUS and using 4 different learning models (2 different learning models on Fashion-MNIST). The entire training dataset was divided between the selected clients in each iteration based on number of clients and IID factor. An IID factor of 1.0 resulted in each client receiving a slice of data with all possible labels. An IID factor of 0.0 resulted in different clients receiving training data with some labels and none receiving training data with all labels. The method for assigning training data based on number of training clients and IID factor is detailed in published literature [3,36]. The malicious ratio determined how clients were trained to act as malicious clients by flipping training labels, and using stealth metrics along with boosting.

Stochastic Gradient Descent was used at each client as the training algorithm with supplied parameters such as number of training epochs, batch size and learning rate.

All experiments were run with **40 iterations**. The default values for all experiments were as follows: **Number of Clients:100, IID Factor:0.4, Malicious Ratio:10%**.

B. Datasets

1) Fashion-MNIST dataset

This well-known dataset consists of 28x28 sized images of fashionable clothing of 10 different types: Trousers, pullover, T-shirt, sneaker, bag, ankle boot, dress, coat, sandal, dress. The images are labelled, making them suitable for classification tasks. The total number of pixels in each image is 784 (28x28), and each pixel contains a light or dark value between 0 and 255. The total number of samples for training is 60,000 images in this dataset, and an additional 10,000 images are available as a test set.

2) CIFAR-10 dataset

The CIFAR-10 is another well-known dataset that consists of 32x32 sized images of transport vehicles and animals of 10 different types: airplane, automobile, ship, truck, bird, cat, deer, dog, frog and horse. The images are labelled, making them suitable for classification tasks. The total number of pixels in each image is 1024 (32x32) and each pixel contains a color value. The total number of samples for training is 50,000 images and an additional 10,000 images are available as a test set. The test batch contains 1,000 images randomly selected from each class. The training batches contain 5,000 images from each class.

3) Adult Census dataset

The Adult Census dataset is a well-known dataset based on 1994 census data. It is labelled and suitable for classification tasks. There are two output labels of whether annual income is >50K or < 50K based on the census data. Each data instance contains 14 features: age, education, education-num, marital status, occupation, relationship, race, sex, capital gain, capital loss, final weight, hours per week, native country and work class. The dataset consists of 45,222 instances, out of which 30,162 form the training set and 15,060 form the test set.

C. Learning Models

Experiments were conducted on four different learning models as described below:

1) Model 1 - Linear Regression Model (FMNIST)

- a) Flatten Layer-Flattens 28x28 images into 784 neurons
- b) Dense Output Layer-10 fully connected neurons for 10 labels

2) Model 2 - 6-layer DNN Model (FMNIST)

- a) Convolution Layer with ReLU activation-64 filters of size 5x5
- b) Convolution Layer with ReLU activation-64 filters of size 5x5
- c) Dropout Layer-Randomly deactivates 25% of neurons
- d) Dense Layer + ReLU activation-128 fully connected neurons
- e) Dropout Layer-Randomly deactivates 50% of neurons
- f) Output Dense Layer -10 fully connected neurons for 10 labels

3) Model 3 - 6-layer DNN Model (CENSUS)

- a) Input Layer-Receives data with 14 features
- b) Dense Layer + ReLU activation-256 fully connected neurons
- c) Dropout Layer-Randomly deactivates 50% of neurons
- d) Dense Layer + ReLU activation-256 fully connected neurons
- e) Dropout Layer-Randomly deactivates 50% of neurons
- f) Output Dense Layer-2 fully connected neurons for 2 output labels

4) Model 4 - 10-layer DNN Model (CIFAR-10)

- a) Conv2D layer - size 64, kernel size 5, strides 2, padding- Inputs images of size 32x32
- b) Batch Normalization + Leaky ReLU activation

- c) Conv2D layer - size 128, kernel size 5, strides 2, padding
- d) Batch Normalization + Leaky ReLU activation
- e) Conv2D layer - size 256, kernel size 5, strides 2, padding
- f) Batch Normalization + Leaky ReLU activation
- g) Conv2D layer - size 512, kernel size 5, strides 2, padding
- h) Batch Normalization + Leaky ReLU activation
- i) Flatten Layer
- j) Output Dense Layer-10 fully connected neurons for 10 labels

D. Performance Metrics

The performance of the defense against the poisoning attacks is measured by the following parameters: Global Accuracy, Precision, Recall, F-1 Score, and Attack Success Rate [36]. A portion of the dataset is reserved as test data and not used for training. After each iteration of client training in the FL process, the global model is used for prediction with test data, and the intermediate results stored in a file. After all iterations are completed (40 in our experiments), performance metrics are evaluated.

The performance metrics use the following measures in the calculations:

True Positive (TP) - malicious samples predicted as malicious
True Negative (TN) - benign samples predicted as benign
False Positive (FP) - benign samples predicted as malicious
False Negative (FN) - malicious samples predicted as benign

1) Global Prediction Accuracy

This is the final accuracy of the global model at the end of all iterations for the particular experiment. It is calculated as the ratio of total correct predictions (both benign and malicious) and the total number of predictions.

2) Attack Success Rate

This metric indicates the success of the malicious attacks. It measures the number of successful malicious attacks with the total number of attempted attacks.

$$ASR = \frac{FN}{TP+FN}$$

3) Precision

This metric is an indication of the performance of the model for malicious predictions and is an important factor while minimizing false positives. This would occur when benign samples are wrongly classified as malicious.

$$Precision = \frac{TP}{TP+FP}$$

4) Recall

This metric, also known as Sensitivity, measures only malicious activity. A high value of this metric indicates more accuracy in malicious detections.

$$Recall = \frac{TP}{TP+FN}$$

5) F1-Score

This metric is a good indication of the overall performance of the model, and is measured as the harmonic mean of the Precision and Recall metrics. F1-Score is a useful metric with imbalanced data.

$$F1-Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

V. EXPERIMENTAL RESULTS

In this section, we discuss the results of our experiments and valuable insights into the performance of the three defense algorithms that are used to protect a FL learning system from malicious actors.

A. Overall Performance Metrics

The overall defense algorithm performance metrics on the three different datasets and four learning models are compared in Table 1. Higher values for all metrics except Attack Success Rate (ASR) indicate better performance. A lower ASR denotes better defense from attacks. The best value for each metric is highlighted in the table for comparison.

B. Varying Clients, IID Factor and Malicious Ratio

The effect on Global Prediction Accuracy by variation of number of clients (20,40,60,80 and 100), IID factor (0.0 -fully non-IID, 0.2,0.4,0.6,0.8 and 1.0 -fully IID) and malicious ratio (10%, 20%, 30% and 40% of all clients) was studied. The experiments were conducted using default values for all other parameters.

Figs. 4, 5, and 6 display the results of the experiments with varying number of clients, IID factor and malicious ratio. Although experiments were conducted for all datasets and learning models, due to similar results, only the results of FMNIST Model 1 are included.

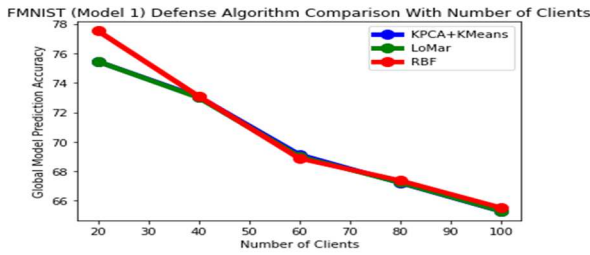


Fig. 4. Comparison of Algorithms with Number of Clients

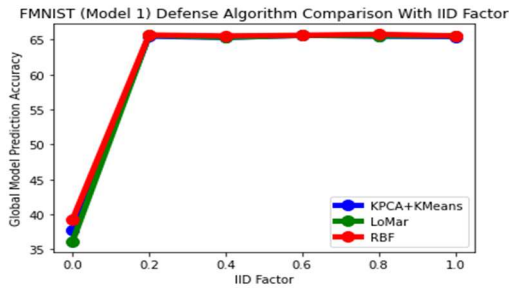


Fig. 5. Comparison of Algorithms with IID Factor

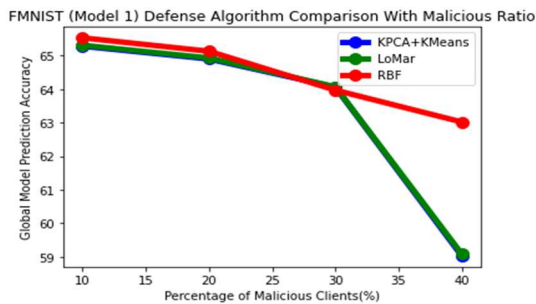


Fig. 6. Comparison of Algorithms with Malicious Ratio

C. Comparison of Execution Times

The execution times for the entire FL learning process after 40 learning iterations using the three defense algorithms was measured. The total execution time in minutes is compared in Table 2. The lowest times for each dataset and learning model is highlighted.

VI. DISCUSSION

The new RBF method uniformly showed excellent results compared to the other existing defense methods for all defense metrics such as Precision, Recall, F1-Score, and Attack Success Rate. The overall Global Prediction Accuracy was also the highest across all datasets and learning models.

The effects of varying the number of clients, the IID factor and malicious ratio were similar with all the compared algorithms, datasets and learning models. This has been verified in other published literature [36] as well. Global Prediction Accuracy drops with increased number of clients, increases with higher IID factor and drops with increased malicious ratio. These are expected results as increasing the number of clients decreases the training that occurs at each client. Increasing the IID factor increases the uniformity of client data, which increases performance. Increasing the number of malicious clients introduces more poisoned data into the system, causing a drop in accuracy.

Table 2 shows a comparison of the total execution times of the compared defense algorithms. LoMar, which involves fine-grained computation by output label, is computationally more intensive. All three techniques are comparable when the model is small (Model 1). RBF showed the lowest execution time with FMNIST Model 2 and CENSUS Model 3. Both these models have intermediate dense layers, increasing the dimensions of the weight matrices. This weight matrix difference affected the total execution time for the other two compared methods more than RBF.

VII. CONCLUSION

This paper describes a new technique called RBF based on Radial Basis Functions for defense of client poisoning attacks in Federated Learning systems. The performance of RBF was compared with two existing defense algorithms based on kernel methods – KPCA with KMeans and LoMar. The algorithms were tested extensively on three different datasets – FMNIST, CENSUS and CIFAR-10 and four different learning models. RBF showed excellent results in all areas, including defense performance metrics, Global Prediction Accuracy and execution times. Based on these results, RBF is a suitable defense against training data attacks in federated learning, even when the data is strongly non-IID.

Future directions for this research would be to experiment with tuning the type of kernel, kernel distance measure, hyperparameters, and malicious outlier detection technique to specific applications and types of attacks for better performance. In addition, most steps in the RBF technique can be parallelized for increased execution speed gain.

Algorithm 1. RBF Malicious Attack Detection at Server

Input: Total number of agents(clients) $\mathbf{K}$, fraction of clients selected at each in each iteration $\mathbf{C}$, total number of training iterations $\mathbf{T}$

Output: Updated global weights at server $\mathbf{G}$

```
1: Initialize global weights  $\mathbf{G}$ 
2: Set  $t=0$  //  $t$  is iteration number
3: while  $t < T$  do
4:   probabilities = CalculateProbabilities( $\mathbf{C} * \mathbf{K}$ )
5:   selectedAgents = SelectAgents(probabilities)
6:   set selectedAgents weights to  $\mathbf{G}$ 
7:   weightUpdates = ParallelTraining(selectedAgents)
8:   for each agent  $i$  in selectedAgents do
9:     for each agent  $j$  in selectedAgents where  $j \neq i$  do
10:      similarityMatrix[ $i$ ][ $j$ ] = ApplyRBFKernel(weightUpdate[ $i$ ], weightUpdate[ $j$ ])
11:    end for
12:  end for
13:  for each agent  $i$  in selectedAgents do
14:    for each agent  $j$  in selectedAgents where  $j \neq i$  do
15:      similarityScore[ $i$ ] = Sum(similarityMatrix[ $i$ ][ $j$ ])
16:    end for
17:  end for
18:  malIndices = DetectMaliciousClients(similarityScore)
19:  for each agent  $i$  in selectedAgents do
20:    if ( $i$  not in malIndices)
21:       $\mathbf{G} = \mathbf{G} + (\text{weightUpdates}[i])$ 
22:    end for
23:   $t=t+1$ 
24: return  $\mathbf{G}$ 
```

TABLE I. OVERALL PERFORMANCE METRICS

Dataset	Defense Algorithm	Global Model Accuracy (%)	Attack Success Rate (%)	Precision (%)	Recall (%)	F1-Score (%)
FMNIST Model 1	KPCA+KMeans	65.29	0	100	100	100
	LoMar	65.31	0	100	100	100
	RBF	65.53	0	100	100	100
FMNIST Model 2	KPCA+KMeans	67.77	5.26	82.87	88.8	84.16
	LoMar	66.34	64.47	25.37	41.6	28.43
	RBF	68.84	3.7	91.92	91.42	91.42
CENSUS Model 3	KPCA+KMeans	75.43	8.8	58.96	88.23	65.5
	LoMar	75.43	2.5	76.96	94.12	79.02
	RBF	75.43	2.15	94.74	94.74	94.74
CIFAR-10 Model 4	KPCA+KMeans	44.59	100	0	0	0
	LoMar	43.55	81.8	4.9	20.5	7.81
	RBF	46.1	0	100	100	100

TABLE II. COMPARISON OF TOTAL EXECUTION TIMES

Dataset	Defense Algorithm	Execution Time(m)
FMNIST Model 1	KPCA+KMeans	4.14
	LoMar	4.38
	RBF	4.16
FMNIST Model 2	KPCA+KMeans	257
	LoMar	282
	RBF	105.75
CENSUS	KPCA+KMeans	9.51
	LoMar	9.86
	RBF	4.5
CIFAR-10	KPCA+KMeans	126.5
	LoMar	135.6
	RBF	128

REFERENCES

- [1] B. McMahan, Eider Moore, D. Ramage, S. Hampson, and B. Aguerre Arcas, "Communication-efficient learning of deep networks from decentralized data". Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, pp. 1273–1282, 2017 (original version on arxiv Feb. 2016).
- [2] P. Kairouz, H.B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, et al. "Advances and open problems in federated learning". Foundations and Trends® in Machine Learning, Vol 14, pp. 1-210, 2021.
- [3] A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, "Analyzing federated learning through an adversarial lens," Proc. Int. Conf. Mach. Learn., pp. 634–643, 2019.
- [4] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data poisoning attacks against federated learning systems," Proc. Eur. Symp. Res. Comput. Secur., pp. 480–501, 2020.
- [5] G. Baruch, M. Baruch, and Y. Goldberg, "A Little Is Enough: Circumventing Defenses For Distributed Learning", NeurIPS, 2019.
- [6] A. N. Bhagoji, S. Chakraborty, S. Calo, and P. Mittal, "Model poisoning attacks in federated learning," Proc. Workshop Secur. Mach. Learn. Collocated 32nd Conf. Neural Inf. Process. Syst., pp. 1–9, 2018.
- [7] M. Fang, X. Cao, J. Jia, and N. Gong, "Local model poisoning attacks to byzantine-robust federated learning," Proc. 29th USENIX Secur. Symp., pp. 1605–1622, 2020.
- [8] P. Blanchard, E.M.E Mhamdi, R. Guerraoui, and J. Stainer. "Machine learning with adversaries: Byzantine tolerant gradient descent", NeurIPS, 2017.
- [9] H. Xiao, B. Biggio, G. Brown, G. Fumera, C. Eckert, F. Roli, "Is feature selection secure against training data poisoning?", International Conference on Machine Learning, pp. 1689–1698, 2015.
- [10] V. Shejwalkar and A. Houmansadr, "Manipulating the Byzantine: Optimizing Model Poisoning Attacks and Defenses for Federated Learning", NDSS, 2021.
- [11] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning." International conference on artificial intelligence and statistics. PMLR, 2020.
- [12] L. Chen, H. Wang, Z. B. Charles, and D. S. Papailiopoulos, "DRACO: Byzantine resilient distributed training via redundant gradients", Proceedings of the 35th International Conference on Machine Learning, 2018.
- [13] E. M. E. Mhamdi, R. Guerraoui, and S. Rouault, "The hidden vulnerability of distributed learning in Byzantium", ICML, 2018.
- [14] D. Alistarh, Z. Allen-Zhu, and J. Li, "Byzantine stochastic gradient descent", NIPS, 2018.
- [15] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates", ICML, 2019.
- [16] W. R. Huang, J. Geiping, L. Fowl, G. Taylor, and T. Goldstein, "Metapoisn: Practical general-purpose clean-label data poisoning," Advances Neural Inf. Process. Syst., vol. 33, pp. 12080–12091, 2020.
- [17] N. Baracaldo, B. Chen, H. Ludwig, and J. A. Safavi, "Mitigating poisoning attacks on machine learning models: A data provenance based approach," Proc. 10th ACM Workshop Artif. Intell. Secur., pp. 103–110, 2017
- [18] Shanmugarasa, Yashothara, et al. "A systematic review of federated learning from clients' perspective: challenges and solutions.", Artificial Intelligence Review, Vol. 56.Suppl 2, 1773-1827, 2023.
- [19] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data poisoning attacks against federated learning systems," Proc. Eur. Symp. Res. Comput. Secur., pp. 480–501, 2020.
- [20] Jebreel, N. Moharram, et al. "LFighter: Defending against the label-flipping attack in federated learning.", Neural Networks, Vol. 170, pp. 111-126, 2024..
- [21] N. Baracaldo, B. Chen, H. Ludwig, and J. A. Safavi, "Mitigating poisoning attacks on machine learning models: A data provenance based approach," Proc. 10th ACM Workshop Artif. Intell. Secur., pp. 103–110, 2017.
- [22] L. Breiman, W. Meisel, and E. Purcell, "Variable kernel estimates of multivariate densities," Technometrics, vol. 19, no. 2, pp. 135–144, 1977.
- [23] B. Tang and H. He, "A local density-based approach for outlier detection," Neurocomputing, vol. 241, pp. 171–180, 2017.
- [24] Wei, Wenqi, and L.Liu. "Trustworthy distributed ai systems: Robustness, privacy, and governance." ACM Computing Surveys, 2024.
- [25] R. H. Byrd, J. Nocedal, and R. B. Schnabel, "Representations of quasi-Newton matrices and their use in limited memory methods", Mathematical Programming, 1994.
- [26] R. Tibshirani, G. Walther, and T. Hastie, "Estimating the number of clusters in a data set via the gap statistic", Journal of the Royal Statistical Society: Series B (Statistical Methodology), 2001.
- [27] S. Park, J. Weimer, and I. Lee, "Resilient linear classification: an approach to deal with attacks on training data", International Conference on Cyber-Physical Systems, pp. 155–164, 2017.
- [28] N. Agarwal, A. Suresh, F. X. Yu, S. Kumar, and B. McMahan, "cpSGD: Communication-efficient and differentially-private distributed SGD", Advances in Neural Information Processing Systems, pp. 7564–7575, 2018.
- [29] R. Agrawal and R. Srikant, "Privacy-preserving data mining", ACM SIGMOD International Conference on Management of Data, 2000.
- [30] X. Li, Z. Qu, S. Zhao, B. Tang, Z. Lu and Y. Liu, "LoMar: A Local Defense Against Poisoning Attack on Federated Learning," in IEEE Transactions on Dependable and Secure Computing, vol. 20, no. 1, pp. 437–450, 1 Jan.-Feb. 2023.
- [31] Z. Zhang, X. Cao, J. Jia, and N. Z. Gong, "FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients.", Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 2545–2555, August 2022.
- [32] X. Cao, M. Fang, J. Liu, and N. Z. Gong. "FLTrust: Byzantine-robust Federated Learning via Trust Bootstrapping", NDSS, 2021.
- [33] A. Samy and S. Girdzijauskas, "Mitigating Sybils in Federated Learning", Proceedings of Information Security Practice and Experience, pp 36-51, ISPEC 2023.
- [34] K. Taejin, et al. "Characterizing Internal Evasion Attacks in Federated Learning." International Conference on Artificial Intelligence and Statistics. PMLR, 2023
- [35] P. Liu, X. Xiu, and W. Wang, "Threats, attacks and defenses to federated learning: issues, taxonomy and perspectives", Cybersecurity, 2022.
- [36] I. Carvalho, K. Huff, L. Gruenwald, J. Bernardino, "Federated Learning: A Comparative Study of Defenses Against Poisoning Attacks", Appl. Sci. 2024, 14, 10706.